



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Algoritmos e Estrutura de Dados II

Anotações de Aula

Profa Melise Maria Veiga de Paula

Tipo Abstrato de Dados

Quando se manipula uma informação no computador, é necessário considerar que esta informação passa por alguns processos de transformação. Veja a figura 1. Num primeiro momento, a informação é representada como um conjunto de dados (processo (1)). O programador precisa definir exatamente qual a estrutura correspondente para uma determinada informação. Por exemplo, se é necessário calcular a média de um conjunto de 100 valores numéricos, o programador pode usar um vetor para representar os dados que correspondem a esta informação.

Tendo representado os dados de uma determinada informação, em algum momento, esses dados são transformados em códigos binários para serem efetivamente manipulados pela máquina (processo (2)). Note que, o processo (2) é transparente para o programador (na maioria das vezes).



Figura – Transformação da Informação

Veja que esta escala de processos é extremamente útil. Manipular a informação diretamente como código binário não é muito produtivo. Neste ponto, as linguagens de programação evoluíram de forma “aproximar” o dado do programador, ainda que, este dado continue sendo manipulado em memória como sendo um conjunto de códigos binários.

Para facilitar esta representação dos dados em memória foram criados três conceitos: tipo de dados, estrutura de dados e tipo abstrato de dados.

Um **tipo de dado** consiste na definição do conjunto de valores (denominado domínio) que uma variável pode assumir ao longo da execução de um problema e um conjunto de operações que podem ser executadas. Podem ser classificados como: tipo básico (primitivo) ou estruturado.

Quando a informação a ser representada é simples, por exemplo, a idade de uma pessoa, um tipo de dado primitivo é suficiente (em C, o tipo int seria um exemplo). Se a informação possui uma estrutura que pode ser definida de forma consistente, é mais útil usar tipos estruturados (em C, podemos usar o struct).

Algumas linguagens, como C, ainda disponibilizam a opção para criação de novos tipos a partir dos tipos básicos ou estruturados. Estes novos tipos são chamados de tipos definidos pelo usuário. Em C, o comando typedef é usado com este objetivo.



MINISTÈRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Por exemplo, considere a criação de dois tipos novos: tIdade (tipo baseado no tipo inteiro) e tProduto (tipo baseado em um struct contendo o código do produto (int) e o nome (cadeia de caracter de tamanho máximo 100)).

Em C, esses novos tipos poderiam ser criados assim:

```
typedef int tIdade;
typedef struct {
    int código;
    char nome[100];
} tProduto;
```

Tendo criado os dois tipos, poderíamos declarar variáveis como sendo do tipo tIdade e do tipo tProduto:

```
tIdade id; tProduto p;
```

Outro conceito muito importante em programação é o termo Estrutura de Dados.

Estrutura de dados são formas genéricas de se estruturar uma informação de modo a serem registradas e processadas pelo computador. O uso de uma estrutura de dados requer o entendimento de dois aspectos: a definição da estrutura e as operações.

Considere a estrutura de dados vetor:

- Definição: conjunto contíguo de dados do mesmo tipo (homogêneo) com tamanho pré-definido. Não é adequado para representar conjuntos que não possuem um tamanho pré-definido (estimado).
- Operações: inserir um elemento no vetor, remover um elemento do vetor e alterar o valor de um elemento. Cada operação é implementada de acordo com os recursos da linguagem de programação usada.

Agora, considere o seguinte problema: você precisa desenvolver uma aplicação que manipula um conjunto de dados sobre seu cliente. Os dados são: nome, idade e identidade. Além disso, de acordo com sua aplicação, você vai precisar incluir e excluir um cliente, alterar e consultar as informações de um determinado cliente, considerando sua identidade.

Analisando as informações, é possível perceber que os dados disponíveis estão relacionados. Será que não podemos criar um tipo para representar estes dados?

Este tipo poderia se chamar tCliente e deveria ter as seguintes informações:

Tipo tCliente {

Nome

Idade

Identidade

}

Operações {

incluirCliente(nome, idade, identidade)

excluirCliente(identidade)

consultarCliente(identidade)



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
    alterarCliente(identidade)
}
```

Repare que, na definição do tipo (dados e operações), não foi considerada a implementação de cada operação. Voltando a pergunta: Será que não podemos criar um tipo para representar estes dados?

A resposta é sim, podemos criar um tipo para representar um cliente. Este tipo é o que chamamos de tipo abstrato de dados (TAD).

Formalmente, um TAD é uma coleção bem definida de dados a serem armazenados, e um grupo de operadores que podem ser aplicados para manipulação desses dados. Um tipo abstrato de dados é um modelo abstrato do armazenamento e manipulação de um determinado conjunto de dados.

Um TAD pode ser usado para representar uma informação que não pode ser declarada usando os tipos primitivos das linguagens. De modo errado, muita gente associa a palavra Abstrata a esta característica. Mas na verdade, a palavra abstrata aqui representa a possibilidade de: uma vez tendo sido definido um TAD e implementado as operações especificadas, qualquer aplicação pode usar este tipo sem se preocupar com a implementação das suas operações, ou seja, a aplicação abstrai “o como” uma operação é executada, basta que a aplicação entenda “o que” cada operação faz.

Para isto, a implementação de cada operação de um TAD deve ser realizada separada da sua especificação. Mas o que é a especificação de um TAD?

Basicamente, a especificação deve conter: o nome do TAD, os dados que ele representa e suas operações. No que se refere às operações, basta definir sua assinatura, ou seja, o nome de cada operação e os dados necessários para sua execução. As informações que devem compor a assinatura de uma operação dependem do ambiente que está sendo considerado para programar.

Mas como usar este conceito na prática?

Na fase de projeto do algoritmo, você define a especificação do TAD (dados e as operações). Na fase de codificação, você define como os dados serão representados e implementa as operações. A implementação depende da linguagem de programação. A idéia básica é que, uma aplicação que usa um TAD é um cliente do TAD. O cliente não precisa conhecer a implementação das operações. Vamos ver como isso pode ser feito na linguagem C.

Em C, existe o conceito de bibliotecas. Estes arquivos possuem extensão .h e contém apenas as assinaturas das operações que são criadas para um determinado tipo de dados abstrato. Além disso, caso o dado a ser utilizado seja estruturado, deve-se informar qual estrutura será utilizado. Em C, uma assinatura deve conter o tipo de retorno da função, o nome da função e os parâmetros de entrada. Já um dados estruturado é definido a partir de uma struct.



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Quando, em um programa, você inclui um arquivo com extensão .h, você está liberado para usar todas as operações especificados na biblioteca. Veja um exemplo:

Suponha que vamos desenvolver um programa que manipule informações sobre pontos em um espaço bidimensional contendo duas coordenadas x e y . Neste programa, precisamos definir algumas operações que serão necessárias para a manipulação do ponto. As operações são:

- criar um ponto dado os valores das coordenadas x e y
- alterar as coordenadas x e/ou y de um ponto
- imprimir as coordenadas de um ponto
- liberar o ponto

Desta forma, temos os dados (o ponto – coordenadas x e y) e as operações necessárias para a definição de um novo tipo que vamos chamar de `tadponto`. O primeiro passo é pensar na interface, que deve conter a especificação do `tadponto`?

Quando um TAD usa um struct, a struct não é definida dentro do arquivo “.h”. Contudo, é necessário informar que a struct poderá ser usada pelos clientes deste TAD. Neste momento, pode-se usar o typedef para criar um “apelido” para struct. Contudo, a definição da struct deve ser definida em outro arquivo junto com a implementação das funções. Lembrando que a interface também deve conter as assinaturas das funções. Portanto, o `tadponto.h` deve ser criado como no código da Figura 1:

```
typedef struct ponto tponto; //informa que a struct ponto que está sendo chamada de
tponto poderá usada pelos clientes do tadponto
tponto *criaponto(int x, int y) ;
void alteraponto(int x, int y, tponto *p);
void libera(tponto *p);
void imprime(tponto *p);
```

Figura 1 – arquivo `tadponto.h` (interface do TAD ponto)

Agora, temos que definir a struct `ponto` e elaborar a implementação das operações do novo tipo. Isto deve ser feito separado da interface do tipo (arquivo .h). Lembre-se que a interface contém apenas a assinatura das operações (cabeçalho das funções). Vamos criar um arquivo com nome `tadponto.c` que deve conter o código apresentado na Figura 2.

Sempre que você tem um .c que implementa funções declaradas em um .h, você deve incluir esse .h no .c. Lembre-se de que o arquivo .h declara as funções, e o arquivo .c implementa. Essa estratégia garante que as funções implementadas no arquivo .c estão compatíveis com as assinaturas definidas no arquivo .h.



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
#include "tadponto.h"
#include <stdio.h>
#include <stdlib.h>
struct ponto {
    int x;
    int y;
};
tponto *criaponto(int x, int y) {
    tponto *p;
    p = malloc(sizeof(tponto));
    if (p) {
        p->x = x;
        p->y = y;
    }
    return p;
}
void alteraponto(int x, int y, tponto *p) {
    p->x = x;
    p->y = y;
}
void libera(tponto *p) {
    free(p);
}
void imprime(tponto *p) {
    printf("%d %d\n", p->x, p->y);
}
```

Figura 2 – Arquivo tadponto.c (implementação do TAD)

Depois de criar a interface e a implementação do TAD, você pode começar a usá-lo no seu programa. Vamos criar a função principal para usar o tadponto. Nosso objetivo será criar um ponto com valores digitados pelo usuário e imprimir esses valores na tela. Depois disso, vamos solicitar novos valores e imprimir os valores alterados na tela. Finalmente, vamos liberar o espaço alocado pelo ponto. Veja o código na Figura 3.



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
#include <stdio.h>
#include "tadponto.h"
int main(void) {
    int x, y;
    tponto *p; /* a função main só enxerga o tadponto.h, neste arquivo não tem
a definição da estrutura, portanto, a main não conhece os campos do tponto,
por isso, p não pode ser um tponto, devendo ser um ponteiro */

    printf("Digite os valores para criar um ponto: \n");
    scanf("%d %d", &x, &y);
    p = criaponto(x, y);
    if (!p) {
        printf("Falha ao criar ponto\n");
        return 1;
    }
    imprime(p);
    printf("Digite os novos valores do ponto:\n");
    scanf("%d %d", &x, &y);
    alteraponto(x, y, p);
    imprime(p);
    libera(p);
    return 0;
}
```

Figura 3 – Função principal – Cliente do tadponto